

■ Day 7 — Materialized Views & Data Sharing

■ Topics Covered

- Materialized Views (MV)
 - Incremental Refresh
 - Standard View vs MV
 - Query Rewrite Optimization
 - Data Sharing
 - Cross-Cluster Analytics
 - Dashboard Optimization
 - Real Enterprise Scenarios
-

■ Real Industry Problem

Heavy dashboard queries may run thousands of times daily.

Each execution scans large tables and recalculates aggregations.

This causes:

- high compute usage
- slow dashboards
- expensive analytics

■ What is Materialized View?

Materialized View stores precomputed query results physically inside Redshift.

Unlike normal views, Materialized Views improve query performance significantly.

■ LAB 1 — Create Base Table

```
CREATE TABLE transactions (  
    transaction_id BIGINT,  
    customer_id BIGINT,  
    city VARCHAR(50),  
    amount DECIMAL(10,2),  
    transaction_time TIMESTAMP  
)  
DISTSTYLE KEY  
DISTKEY(customer_id)  
SORTKEY(transaction_time);
```

■ Insert Sample Data

```
INSERT INTO transactions VALUES  
(1,101,'Mumbai',1200,'2026-07-01 10:00:00'),  
(2,102,'Pune',5000,'2026-07-01 11:00:00'),  
(3,101,'Mumbai',800,'2026-07-01 12:00:00'),  
(4,103,'Delhi',9000,'2026-07-01 13:00:00');
```

■ LAB 2 — Create Materialized View

```
CREATE MATERIALIZED VIEW mv_customer_spending AS
SELECT
    customer_id,
    SUM(amount) total_spent
FROM transactions
GROUP BY customer_id;
```

■ LAB 3 — Query Materialized View

```
SELECT *
FROM mv_customer_spending;
```

■ Incremental Refresh

New transactions continuously arrive in production systems.

Materialized Views must be refreshed to reflect latest data.

Incremental refresh processes only changed/new rows instead of recomputing full datasets.

■ LAB 4 — Refresh MV

```
INSERT INTO transactions VALUES
(5,101, 'Mumbai', 3000, '2026-07-01 14:00:00');

REFRESH MATERIALIZED VIEW mv_customer_spending;

SELECT *
FROM mv_customer_spending;
```

■ Query Rewrite Optimization

Redshift optimizer may automatically redirect queries to Materialized Views.

This avoids scanning large base tables repeatedly.

```
SELECT
    customer_id,
    SUM(amount)
FROM transactions
GROUP BY customer_id;
```

■ View vs Materialized View

Normal View:

- no physical storage
- recalculates every query

Materialized View:

- stores data physically
- much faster queries
- optimized for dashboards

■ Redshift Data Sharing

Allows live data sharing between Redshift clusters/accounts without copying data.

Benefits:

- no duplicate storage
- real-time access
- multi-team analytics

■ LAB 5 — Create Datashare

```
CREATE DATASHARE sales_share;  
  
ALTER DATASHARE sales_share  
ADD TABLE transactions;  
  
GRANT USAGE  
ON DATASHARE sales_share  
TO NAMESPACE 'consumer-namespace-id';
```

■■ Real Enterprise Architecture

Raw Data → Amazon S3

↓

Spectrum Queries

↓

Redshift Gold Tables

↓

Materialized Views

↓

Data Sharing

↓

BI / ML / Analytics Teams

■ LAB 6 — Dashboard Optimization MV

```
CREATE MATERIALIZED VIEW mv_city_revenue AS  
SELECT  
    city,  
    SUM(amount) total_revenue  
FROM transactions  
GROUP BY city;  
  
SELECT *  
FROM mv_city_revenue;
```

■ Real Production Challenges

- stale Materialized Views
- refresh overhead
- duplicate analytics pipelines
- slow dashboard queries

Solutions:

- scheduled refresh
- incremental refresh
- data sharing
- pre-aggregated MVs

■ Important Interview Questions

Q. What is Materialized View?

A. Stores precomputed query results physically for faster analytics.

Q. Why faster than normal view?

A. Avoids repeated aggregation calculations.

Q. What is Incremental Refresh?

A. Refreshing only changed/new rows.

Q. What is Redshift Data Sharing?

A. Live data sharing without copying datasets.

■ Day 7 Outcome

■ Learned Materialized Views

■ Learned Incremental Refresh

■ Learned Query Rewrite Optimization

■ Learned Data Sharing

■ Learned Dashboard Optimization

■ Learned Enterprise Analytics Architecture